# Assignment 6 Visitor Pattern

Advanced File System | Software Design Patterns

Yelnur Pashanov | Software Engineering | Year 2 | 8 November 2026

Repository: https://github.com/aokoshi/SDP-Assignment6

## 1 Purpose and object model

The project implements an in-memory file system with three independent operations: total size, configurable file search and tree statistics. Java 17, Maven and JUnit 5 are used. The same tree is processed by all three visitors without putting operation-specific logic into the elements.

Element defines name() and accept(Visitor). FileNode stores an immutable name and non-negative byte size. DirectoryNode owns named children. The package-private abstract Node stores the shared name and parent reference. DirectoryNode.addNode checks ownership, ancestor cycles and sibling names before changing the tree; invalid changes throw IllegalArgumentException. Null arguments are rejected with NullPointerException. Children are exposed through an immutable List.copyOf snapshot.

## 2 Double dispatch and traversal

First, child.accept(visitor) dynamically selects FileNode.accept or DirectoryNode.accept. Inside that concrete method, visitor.visit(this) selects the overload for the known element type, then dynamic dispatch selects the concrete visitor implementation. For example, a FileNode accepted by TotalSizeVisitor reaches visit(FileNode), which adds its bytes. No instanceof, reflection, getClass-based branching or behavior-selecting casts are used.

Visitors own a depth-first traversal: each directory overload calls accept on its children exactly once. Element accept methods do not recurse. The tree invariants prevent shared children and cycles, so every reachable element is visited once per operation. This keeps the element contract small, but repeats the short traversal loop in each visitor. Nesting has no fixed application limit; very deep trees remain limited by the Java call stack.

calculate, search and collect reset their state at the start of every run; use these entry points for complete operations. Visitor instances can be reused sequentially, but are not thread-safe. Search returns an immutable result snapshot. TotalSizeVisitor uses Math.addExact so a sum outside long range fails explicitly.

## 3 Five Clean Code practices

**Meaningful names.** SearchVisitor.minimumSize and StatisticsVisitor.maxDepth state the meaning of their values directly.

**Single responsibility.** FileNode stores file data; TotalSizeVisitor.calculate owns the size operation, so changing that operation does not change the element.

**Small methods.** FileNode.accept is one delegation and StatisticsVisitor.recordDepth performs one calculation, making the dispatch and depth logic easy to trace.

**Encapsulation.** DirectoryNode.children returns a read-only snapshot and addNode validates changes, so callers cannot bypass invariants through the child list.

**No type-checking chains.** Visitor.visit overloads and both accept methods select behavior through polymorphism instead of conditional checks on element types.

## 4 Demonstration and deterministic results

Main creates workspace with notes.txt (120 bytes), docs/guide.txt (300), docs/archive/old.txt (80), and an empty directory named empty. The search criterion is name.endsWith(".txt") with an inclusive threshold of 100 bytes.

```
mvn test
mvn compile exec:java
```

```
Total bytes: 500
Text files >= 100 bytes: [docs/guide.txt, notes.txt]
Files: 3, directories: 4, max depth: 3
```

Search combines filename and size using AND. It excludes the chosen directory root name, builds paths with / and sorts them lexicographically. A file chosen as the root is represented by its filename. Statistics count the directory root; depth starts at 0 and increases by one for each child. Therefore old.txt has depth 3. Empty directories count as directories and contribute zero bytes.

## 5 Automated verification

Test class: edu.aitu.visitor.VisitorTest. A fresh build using Java 17.0.20.1 ran mvn clean test exec:java successfully on 8 November 2026. Result: 10 tests, 0 failures, 0 errors, 0 skipped. The demonstration above is the actual program output.

| ID | Test method in VisitorTest | Verified behavior |
|---|---|---|
| T1 | doubleDispatchCallsCorrectOverloads | Both concrete element overloads |
| T2 | sizeIncludesEveryNestedFile | Four nested files total 69 bytes |
| T3 | searchCombinesNameAndInclusiveThreshold | Name AND size; boundary size 10 |
| T4 | searchSortsRelativePathsIndependentlyOfInsertionOrder | Sorted relative paths and a file root |
| T5 | statisticsIncludeRootAndDeepestFile | 4 files, 4 directories, depth 3 |
| T6 | emptyDirectoryAndNoMatchesAreHandled | Empty root and no search matches |
| T7 | repeatedRunsResetAllVisitors | Same instances, changing roots |
| T8 | invalidTreeChangesAreRejected | Sizes, duplicate names, parents, cycles |
| Extra | sizeOverflowIsExplicit; namesAndThresholdAreValidated | Overflow and invalid inputs |

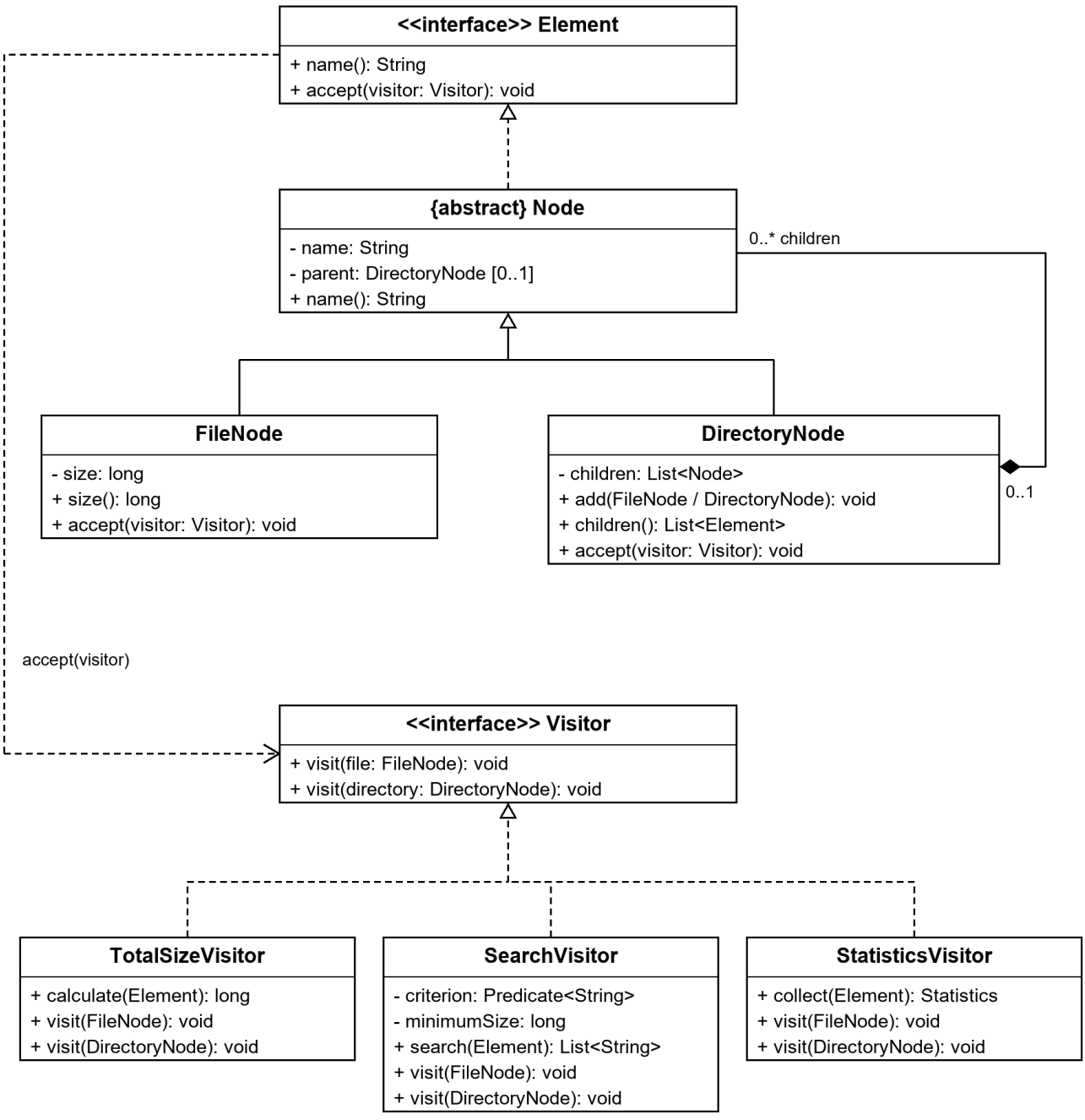
## 6 Extension boundary and trade offs

Commit 6728a3c introduced the elements, Visitor contract, size and search visitors. Commit ba40f56 added StatisticsVisitor, tests and Main without changing FileNode, DirectoryNode or either accept method. A new operation needs a new visitor implementation only. A new element type needs another Visitor overload and updates to every existing visitor; the directory insertion API must also be extended. Visitor fits a stable set of element types with growing operations.

Defense example: raise the Main search threshold from 100 to 300. Only docs/guide.txt remains because the boundary is inclusive; total size and statistics stay unchanged. Trace the two dispatch steps in accept, then show the boundary assertion in searchCombinesNameAndInclusiveThreshold.

## UML class diagram

Interfaces, two concrete elements, the shared base class and all three visitors. Only the main fields and operations are shown. Statistics is the nested immutable result record with files, directories and maxDepth.




Solid line + hollow triangle: inheritance | Dashed line + triangle: realization

Filled diamond: composition | Dashed open arrow: dependency